

基于 Multi-Agent 与 MCP 的汽车 4S 店客户流失挽留方案：设计、可观测性与评估实践

作者：Qin Lu

日期：24th Sep 2026

项目：Multi-Agent Customer Retention Workflow

目录

- Solution Design (方案设计)
- Agent Architecture (Agent 架构)
- Observability Strategy (可观测性策略)
- Evaluation Approach (评估方法)
- Workflow & Deployment (工作流与部署)

1. Solution Design (方案设计)

1.1 背景与目标

本方案面向汽车 4S 店 (Vehicle Group 旗下) 的客户流失挽留场景，目标是构建一个 Multi-Agent 工作流：从客户数据出发，自动完成流失风险识别、挽留策略决策、个性化邮件生成，并通过 MCP 工具完成后续动作。

1.2 两个候选设计及选型依据

设计一：TurboApply 通用推荐框架

源于我自研的求职推荐工具 TurboApply，目前以下功能已经用于我的 TurboApply 中，但是我想将其拓展为适合所有人和所有公司的一个市场战略工具，具体流程为：

- 输入目标公司名称；
- Agent 1 生成该公司所属行业、副标题、标签等画像信息；
- Agent 2 基于画像生成 Top-K 最相似公司（求职者 Top 10，企业客户 Top 1 即最大竞争者）；
- 接入 MCP 获取该公司最新相关新闻。

设计二：Multi-Agent Customer Retention Workflow

基于此前的 [Hybrid-ML-LLM-Retrieval-System-for-Customer-Churn-Decisioning](#) 项目衍生而来。原项目以 ML 为主、LLM 为辅，未引入 Agentic AI，但场景天然适合 Multi-Agent 改造：

- 输入与用户流失评估强相关数据特征；
- Agent 1 生成客户流失风险评级；
- Agent 2 生成挽留客户的促销建议；
- Agent 3 生成个性化促销邮件；
- Agent 4 通过 Outlook MCP 定时推送邮件。

选型结论：设计一在创新性与通用性上更强，但实现周期更长；设计二更成熟、可复用已有 ML 资产，且能在 Event 时间内完成端到端验证。因此最终选择**设计二**作为落地方案。

2. Agent Architecture (Agent 架构)

2.1 Agent 角色与职责

表 1: Agent 角色与职责

Agent	职责	输入	输出
churn-triage-agent	客户流失分诊	客户四项 churn 指标	VIN、车辆摘要、churn risk、flagged metrics
churn-decision-agent	促销决策	triage 输出	唯一推荐促销动作 + 理由

Agent	职责	输入	输出
email-content-agent	邮件内容生成	决策输出	Subject + Email body
auto-send-agent (MCP)	工具调用/自动 动作	邮件内容	MCP 工具执行结果

churn-triage-agent 关键设计：

- 四项指标：customer_value_score (越高越好)、churn_risk_score (越低越好)、rfm_loyalty_score (越高越好)、vehicle_usage_lifecycle_score (越低越好)。
- 每个指标有可接受区间 [min, max], 超出即 flagged, deviation 为超出距离。
- 风险分级：🔴 high = 3+ flagged; ⚠️ medium = 1-2; ✅ low = 0。

churn-decision-agent 关键设计：

- 仅推荐一个促销：
 - extended_warranty_insurance: 保修过期/即将过期且里程 < 80,000 km;
 - new_car_trade_in: 里程 ≥ 80,000 km 且车龄 == 4 年;
 - 否则 maintenance_package。
- 若多条命中，选择针对最强 churn 信号的促销。
- 输出格式固定：RECOMMENDED ACTION: ... REASONING: ...

email-content-agent 关键设计：

- 严格基于推荐促销生成，不重新评估推荐；
- 200 词以内，结合车龄、里程、保修状态与 churn 信号个性化；
- 不虚构价格、折扣、截止日期；
- 输出格式：SUBJECT: ... EMAIL: ...

auto-send-agent 的替代方案：

原计划使用 Outlook MCP 自动发送，因时间紧张，改为通用 MCP Agent：

"You are a helpful agent that can use MCP tools to assist users. Use the available MCP tools to answer questions and perform tasks."

最终部署时接入了 azure-rest-api-specs MCP，而非 Outlook 自动发送 MCP。

2.2 Workflow 编排与数据流

text

客户数据（基于 ML 模型得到的与客户流失评估强相关的结果作为这个项目的输入）

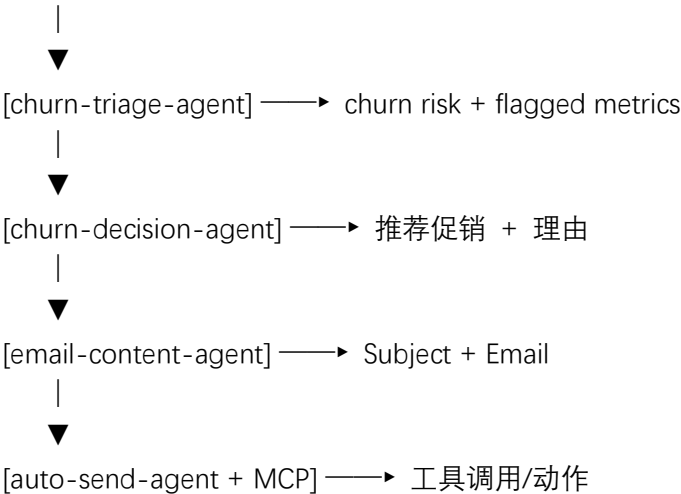
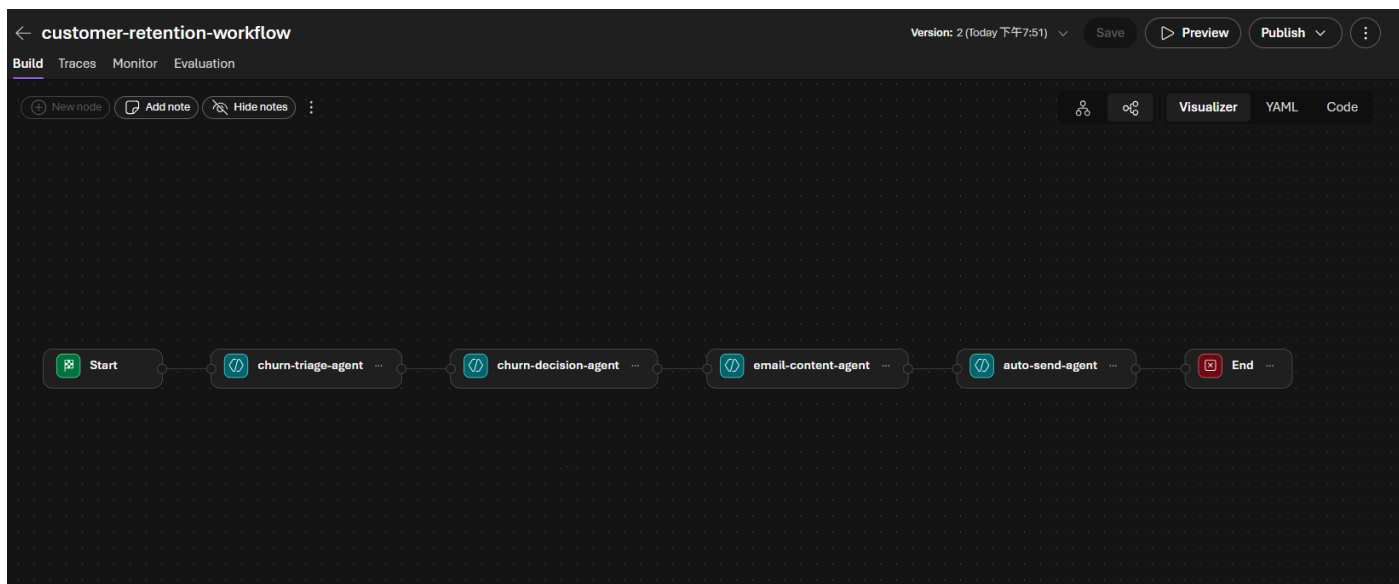


图 1：四 Agent workflow数据流图



2.3 MCP 集成

- Baseline 阶段未接 MCP，仅两个 Agent 串联；
- 进阶阶段融入了后二个 Agent，并且最后一个 agent with MCP，由于时间极其有限，所以最后一个 outlook MCP 改用 azure-rest-api-specs MCP 替换测试整个工作流；
- 创建工作流的时候，相比 Python SDK，在 Foundry Workflow 是直接创建会更加方便。

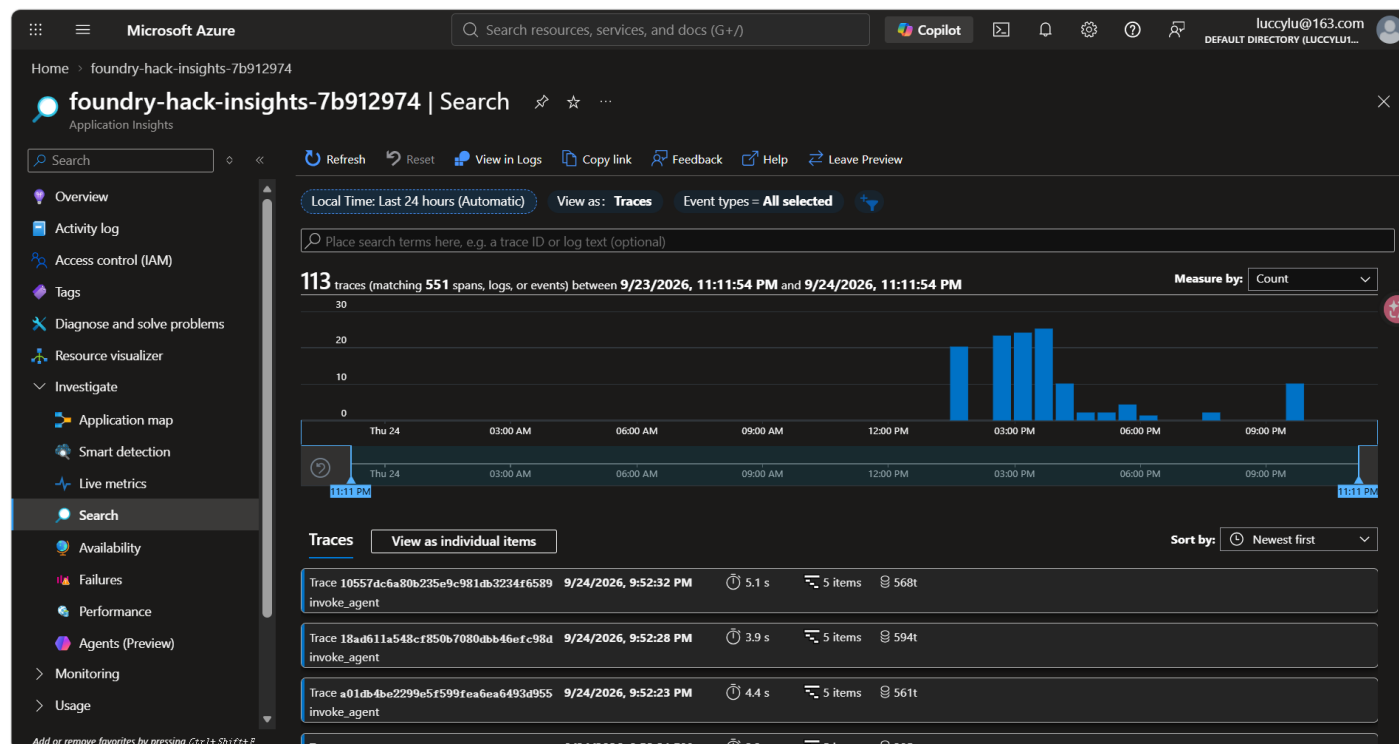
3. Observability Strategy（可观测性策略）

3.1 Application Insights + GenAI Tracing

使用 Application Insights with GenAI tracing 提供端到端可观测性，覆盖：

- Token 使用量；
- 延迟；
- 工具执行；
- 错误追踪。

Figure 2. Application Insights



3.2 Traces / Monitor 中的关键观测维度

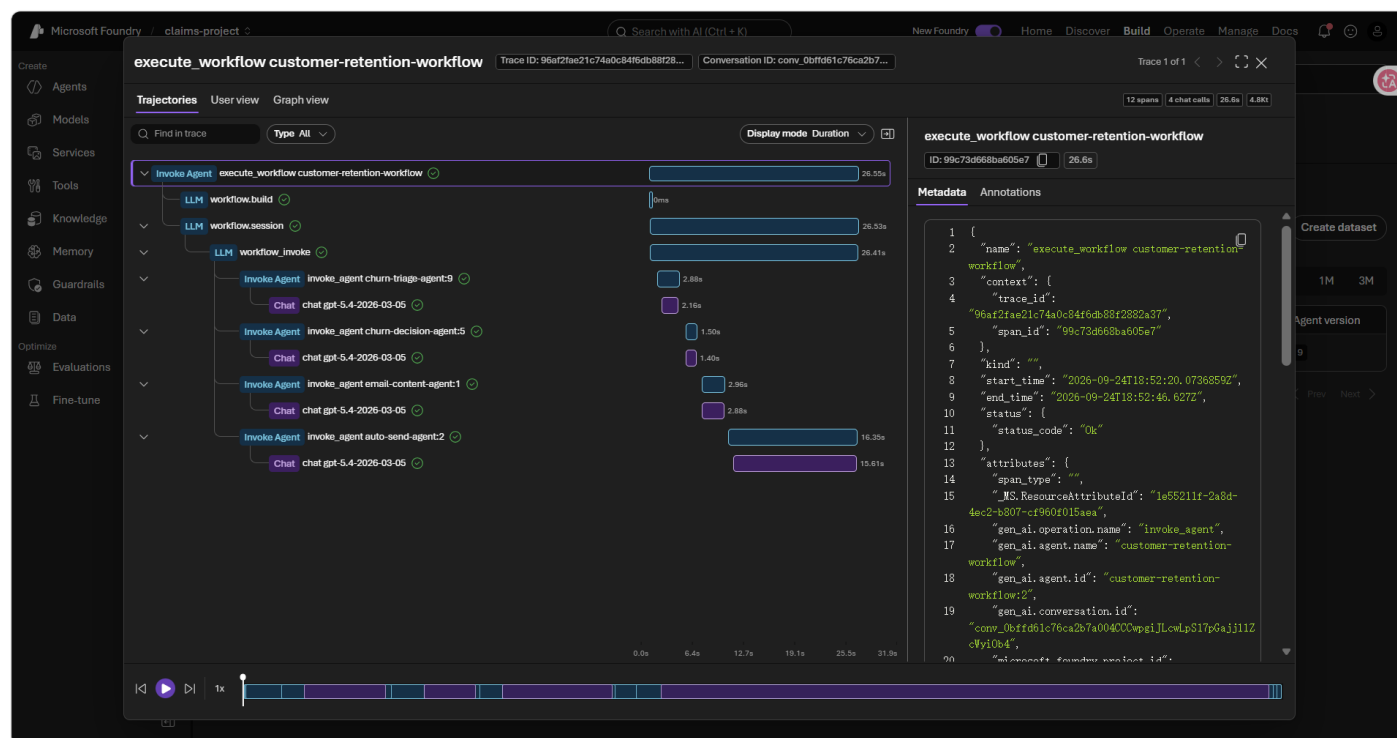
实际主要依赖 Agent 与 Workflow 自带的 Traces 与 Monitor，其数据与说明非常详细，可直接看到：

- 每个 Agent 的 input / output；

- metadata;
- 报错原因。

这一层是后续 Debug 的主要依据。例如 V8 阶段正是通过 Traces 才定位到 Error code: 400 的具体原因。

Figure 3. Traces



4. Evaluation Approach（评估方法）

4.1 Online Evaluation 的核心作用

Online Evaluation 是判断 Agent 设计质量的关键手段。它不仅能评估输出质量，还能暴露数据设计与 Prompt 设计中的结构性问题。

4.2 训练数据 Label 泄漏问题

一个容易被忽视、且很难通过 Online Evaluation 直接发现的问题是 **label 泄漏**：

- 例如输入为 feature1=1, feature2=2, feature3=risk level, 却让 LLM 输出 risk level;
- 等于提前把答案告诉了 LLM, LLM 会直接拿 feature3 作为输出, 评估结果失真;
- 正确做法: 输入只放 feature1=1, feature2=2, 让 LLM 自己推理 risk level。

版本	变动内容	结果
V1	去掉 vehicle info	评估失败
V2	加入 vehicle info	评估失败
V3	优化两个 Agent 的 prompt, 避免结构乱码、表述更简洁	部分成功, 失败 claim 无数据, 无法定位
V4	修改 ground truth 中的转义字符	部分成功, 同上
V5	加入 threshold	部分成功, 同上; 开始怀疑吞吐与非并行导致漏读
V6	将 jsonl 拆分为三个小文件测试	部分成功

版本	变动内容	结果
V7	只测试第一个 VIN customer id	确认非并行问题，而是 query 本身让 LLM 无法推理，属模型层面问题
V8	GPT-6-luna + reasoning effort low	推理失败，但 Traces 显示详细报错：Error code: 400
V9	GPT-6-luna + reasoning effort high	推理失败
V10	GPT-6-astra + reasoning effort high	推理成功
V11	两个 Agent 均使用该模型配置评估全量数据	二个 agent 均全部成功

4.3 Ground Truth 数据集设计原则

不建议把多个 Agent 的 ground truth 混在一起，例如：

json

ground_truth: {churn_risk: high, recommended_action: maintenance_package}

其中 churn_risk 来自 Agent 1, recommended_action 来自 Agent 2。这种做法是否可行取决于评估指标：

- 若使用 **coherence and fluency** 等整体性指标，可以合并 ground truth，再分别评测两个 Agent；
- 若使用 **逐 Agent 精确匹配** 类指标，则应拆分 ground truth，避免相互污染。

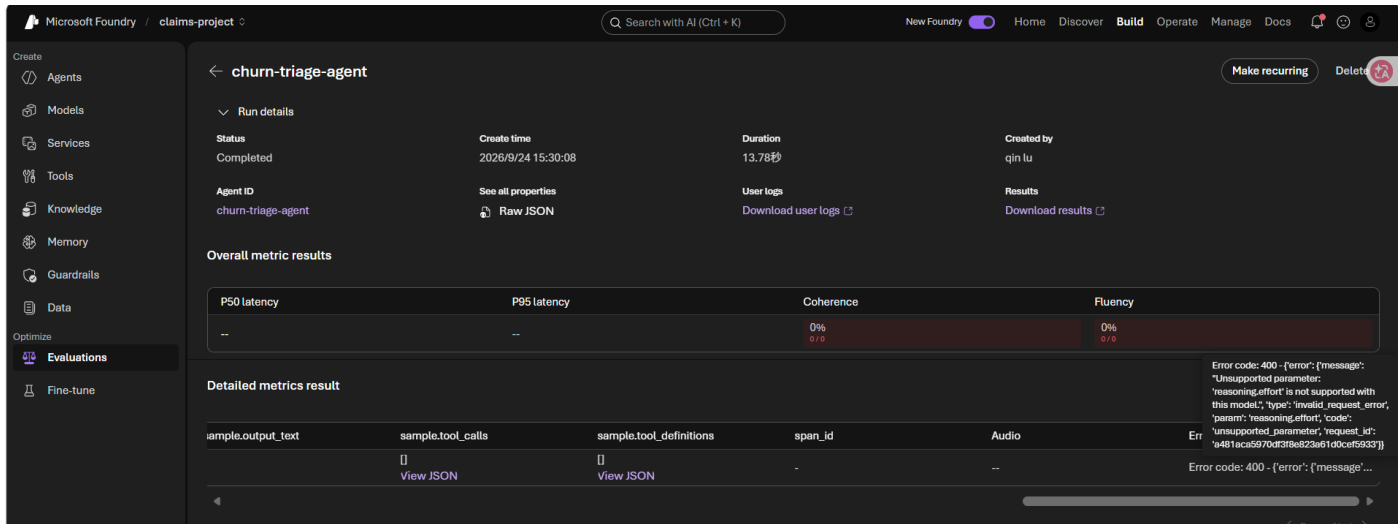
4.4 评估驱动的 Debug 迭代 (V1-V11)

表 2：评估驱动的 Debug 迭代记录

关键结论：

- 失败根因不在并行/吞吐，而在 **模型能力与 query 可推理性的匹配**；
- Traces 是定位模型层错误的关键工具；
- 模型配置（model + reasoning effort）对多 Agent 评估成功率影响显著。

Figure 4. Evaluation detailed metrics result



5. Workflow & Deployment（工作流与部署）

5.1 Python SDK 部署（Baseline）

- 使用 Deploy.py 通过 Python SDK 部署两个 Agent 的 Workflow；
- Baseline 仅含两个 Agent，测试效果良好。

关键注意事项：

- Preview 时输入数据不应包含 risk level 等 ground truth 字段，因为 Workflow 会自动生成 Agent 1 的

结果并传给 Agent 2;

- 在 agents.py 中跑脚本时，虽然结果正确，但两个 Agent 实际是**独立执行的**：Agent 2 的设计本应依赖 Agent 1 的输出，但脚本中 Agent 2 的 recommended action 实际来自 LLM 自身推理，而非上游传递。

5.2 Foundry Workflow 部署（MCP 版本）

- 完成 Baseline 后，开始融入更多 Agent 并接入 MCP;
- 因时间有限，最终未部署 Outlook 自动发送 MCP，而是部署 azure-rest-api-specs MCP;
- 直接通过 **Foundry 创建 Workflow**，比 Python SDK 更便捷高效，测试成功。

5.3 GitHub

完整的实现、源代码、技术文档、评估细节以及相关截图均可在项目仓库中获取：[Multi-Agent Customer Retention Platform — GitHub Repository](#)

该仓库提供了本文档范围之外的更多实现细节和支持材料，包括工作流配置、智能体实现、评估迭代过程以及开发截图。